

# Bases de Datos a Gran Escala

## Sesión 1 · Modelado de datos, modelo relacional y consultas SQL

Diego Sevilla Ruiz

Departamento de Ingeniería y Tecnología de Computadores

Facultad de Informática · Universidad de Murcia

# Modelado de datos

Decidir qué son las cosas, qué debe ser cierto y qué consultas deben quedar optimizadas

# ¿Por qué modelamos?

---

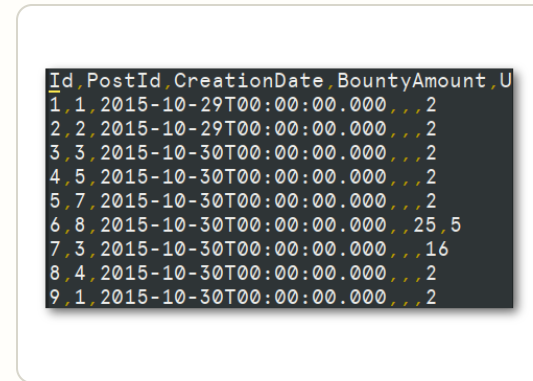
- Tenemos que resolver un **problema**: realizar un desarrollo, sea una aplicación, un servicio o un análisis
- Ese desarrollo necesita **guardar datos**, y esos datos **se relacionan entre sí**
- Los datos tienen que servir al problema: responder a sus preguntas, respetar sus reglas y seguir siendo útiles cuando el desarrollo cambie

**¿Cómo modelamos los datos para que se ajusten al problema que queremos resolver?**

# Los datos no nacen modelados



Directorios y ficheros



Ficheros CSV



Hojas de cálculo



Pares clave/valor

También registros, *streams* y formatos columnares. Llegan sin claves, sin tipos garantizados y sin reglas: la estructura no está en los datos, **la imponemos nosotros** al modelar

# Con qué se modela: entidades y relaciones

| Elemento        | Qué es                                                                                                 | En una biblioteca                  | Cómo se convierte en datos                                                                                                                       |
|-----------------|--------------------------------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entidad</b>  | Algo del dominio con <b>identidad</b> propia: se distingue de las demás aunque se le parezca           | Socio, libro                       | Un conjunto de registros, cada uno con un <b>identificador</b> que no se repite                                                                  |
| <b>Atributo</b> | Una propiedad de una entidad o de una relación                                                         | Nombre del socio, título del libro | Un campo, con un tipo y unos valores válidos                                                                                                     |
| <b>Relación</b> | Una asociación entre entidades, con una <b>cardinalidad</b> : uno a uno, uno a muchos, muchos a muchos | Un socio toma prestado un libro    | Una <b>referencia</b> al identificador de la otra entidad o, si es muchos a muchos o tiene atributos (la fecha del préstamo), un registro propio |

Decidir qué es entidad, qué es atributo y qué es relación es la primera decisión del modelado

# El problema: un sitio de preguntas y respuestas

---

Queremos desarrollar un sitio web como Stack Overflow:

- Los **usuarios** publican **preguntas** y **responden** a las preguntas de otros
- Cualquier usuario puede **comentar** preguntas y respuestas
- Las preguntas y las respuestas reciben **votos** a favor o en contra
- Quien pregunta puede **aceptar** una de las respuestas
- Los votos y las respuestas aceptadas dan **puntuación** a su autor

De esta especificación sacaremos las entidades y las relaciones del modelo

# Un caso concreto

---

El 3 de marzo, Ana preguntó cómo ordenar resultados por fecha. Luis respondió al día siguiente y su respuesta recibió 12 votos. Ana la marcó como aceptada

- ¿Qué **entidades** intervienen y qué **relaciones** las unen?
- ¿«Aceptada» es atributo de la respuesta, de la pregunta o de la relación entre ambas?
- ¿Qué ocurre con la pregunta si Ana borra su cuenta?
- Modelar es decidir esto **antes** de escribir la primera consulta

# Qué aporta un buen modelo

---

- Describe el problema en el **nivel de abstracción adecuado** y descarta lo accesorio
- Decide qué consultas quedan **optimizadas** y cuáles resultan **costosas**: es una decisión de rendimiento, no sólo de orden
- Hace explícitas las reglas del dominio para que el sistema **impida** los estados imposibles
- Permite que varias aplicaciones usen los mismos datos sin negociar cada consulta
- Los datos **sobreviven** a las aplicaciones que los crearon: el modelo es lo que queda cuando cambian el lenguaje, el producto y la moda

# Modelo frente a código



```
-- Schema stackoverflow
--
CREATE SCHEMA IF NOT EXISTS 'stackoverflow' DEFAULT CHARACTER SET latin1;
USE 'stackoverflow';

-- Table 'stackoverflow`.`Users`
--
CREATE TABLE IF NOT EXISTS 'stackoverflow`.`Users` (
  'Id' INT(11) NOT NULL DEFAULT '0',
  'DisplayName' TEXT NULL DEFAULT NULL,
  'Views' INT(11) NULL DEFAULT NULL,
  'DownVotes' INT(11) NULL DEFAULT NULL,
  'LastAccessDate' TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  'ProfileImageUrl' TEXT NULL DEFAULT NULL,
  'WebsiteUrl' TEXT NULL DEFAULT NULL,
  'Reputation' INT(11) NULL DEFAULT NULL,
  'Location' TEXT NULL DEFAULT NULL,
  'Age' INT(11) NULL DEFAULT NULL,
  'UpVotes' INT(11) NULL DEFAULT NULL,
  'CreationDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'AboutMe' TEXT NULL DEFAULT NULL,
  'AccountId' INT(11) NULL DEFAULT NULL,
  PRIMARY KEY ('Id'),
  ENGINE = InnoDB
  DEFAULT CHARACTER SET = utf8;

-- Table 'stackoverflow`.`Posts`
--
CREATE TABLE IF NOT EXISTS 'stackoverflow`.`Posts` (
  'Id' INT(11) NOT NULL DEFAULT '0',
  'ParentId' INT(11) NULL DEFAULT NULL,
  'Body' TEXT NULL DEFAULT NULL,
  'ViewCount' INT(11) NULL DEFAULT NULL,
  'LastEditorDisplayName' TEXT NULL DEFAULT NULL,
  'ClosedDate' TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  'Title' TEXT NULL DEFAULT NULL,
  'LastEditorUserId' INT(11) NULL DEFAULT NULL,
  'LastActivityDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'LastEditDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'AnswerCount' INT(11) NULL DEFAULT NULL,
  'CommentCount' INT(11) NULL DEFAULT NULL,
  'AcceptedAnswerId' INT(11) NULL DEFAULT NULL,
  'Score' INT(11) NULL DEFAULT NULL,
  'OwnerDisplayName' TEXT NULL DEFAULT NULL,
  'PostTypeId' INT(11) NULL DEFAULT NULL,
  'OwnerUserId' INT(11) NULL DEFAULT NULL,
  'Tags' TEXT NULL DEFAULT NULL,
  'CreationDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'FavoriteCount' INT(11) NULL DEFAULT NULL,
  'CommunityOwnedDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  PRIMARY KEY ('Id'),
  INDEX 'ParentId' ('ParentId') ASC,
  INDEX 'OwnerUserId' ('OwnerUserId') ASC,
  INDEX 'AcceptedAnswerId' ('AcceptedAnswerId') ASC,
  CONSTRAINT 'Posts_ibfk_1'
    FOREIGN KEY ('ParentId')
      REFERENCES 'stackoverflow`.`Posts` ('Id'),
  CONSTRAINT 'Posts_ibfk_2'
    FOREIGN KEY ('OwnerUserId')
      REFERENCES 'stackoverflow`.`Users` ('Id'),
  CONSTRAINT 'Posts_ibfk_3'
    FOREIGN KEY ('AcceptedAnswerId')
      REFERENCES 'stackoverflow`.`Posts` ('Id'),
  ENGINE = InnoDB
  DEFAULT CHARACTER SET = utf8;

-- Table 'stackoverflow`.`Comments`
--
CREATE TABLE IF NOT EXISTS 'stackoverflow`.`Comments` (
  'Id' INT(11) NOT NULL DEFAULT '0',
  'UserId' INT(11) NULL DEFAULT NULL,
  'PostId' INT(11) NULL DEFAULT NULL,
  'Score' INT(11) NULL DEFAULT NULL,
  'CreationDate' TEXT NULL DEFAULT NULL,
  'UserDisplayName' TEXT NULL DEFAULT NULL,
  'Text' TEXT NULL DEFAULT NULL,
  PRIMARY KEY ('Id'),
  INDEX 'PostId' ('PostId') ASC,
  INDEX 'UserId' ('UserId') ASC,
  CONSTRAINT 'Comments_ibfk_1'
    FOREIGN KEY ('PostId')
      REFERENCES 'stackoverflow`.`Posts` ('Id'),
  CONSTRAINT 'Comments_ibfk_2'
    FOREIGN KEY ('UserId')
      REFERENCES 'stackoverflow`.`Users` ('Id'),
  ENGINE = InnoDB
  DEFAULT CHARACTER SET = utf8;
```





«Show me your flowcharts and conceal your tables, and I shall continue to be mystified. Show me your tables, and I won't usually need your flowcharts; they'll be obvious.»

— Fred Brooks, *The Mythical Man-Month*, 1975

Cincuenta años después sigue siendo el mejor resumen de por qué empezamos por el modelo

# ¿Qué es un modelo de datos?

---

Todo modelo de datos define tres componentes:

- 1. Estructuras** para albergar los datos
- 2. Restricciones** que deben cumplir
- 3. Operaciones** que se pueden realizar

# Ejemplos de los tres componentes

---

| Componente    | Preguntas que responde                                                                                                         |
|---------------|--------------------------------------------------------------------------------------------------------------------------------|
| Estructuras   | ¿Filas y columnas? ¿Vértices y aristas? ¿Pares clave/valor?<br>¿Una secuencia de bytes?                                        |
| Restricciones | ¿Todas las filas tienen las mismas columnas? ¿Qué valores admite un atributo? ¿Puede un archivo tener dos padres?              |
| Operaciones   | ¿Cuál es el valor asociado a la clave <code>x</code> ? ¿Qué filas contienen «plaza»? ¿Cuáles son los siguientes $N$ registros? |

Haremos estas tres preguntas a cada sistema del curso

# Los tres componentes en el ejemplo

---

| Componente    | En el foro de preguntas y respuestas                                                                                |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Estructuras   | Usuario , Pregunta , Respuesta , Comentario , Voto ;<br>«aceptada» es una referencia de la pregunta a una respuesta |
| Restricciones | Cada respuesta es de una sola pregunta; como mucho una aceptada, y debe ser suya                                    |
| Operaciones   | Una pregunta con sus respuestas por votos; las preguntas de Ana por fecha; quién responde más                       |

Si Ana borra su cuenta, el modelo decide: borrar sus preguntas, conservarlas sin autor o impedirlo. **Stack Overflow las conserva**

# Tres niveles de decisión

---

| Nivel      | Qué decide                          | En el ejemplo                                   |
|------------|-------------------------------------|-------------------------------------------------|
| Conceptual | Qué cosas y relaciones existen      | Usuarios que preguntan, responden y votan       |
| Lógico     | Cómo se expresan en un paradigma    | Tablas, documentos, filas por partición o grafo |
| Físico     | Cómo se guardan y qué se lee rápido | Índices, orden en disco, particiones, réplicas  |

En el modelo relacional el nivel físico cambia sin tocar las consultas: es la **independencia física**

# Los mismos hechos, cuatro modelos

| Modelo · sesión                  | Cómo guarda los hechos                                       | Consultas optimizadas                     | Consultas costosas                                |
|----------------------------------|--------------------------------------------------------------|-------------------------------------------|---------------------------------------------------|
| <b>Relacional</b> · 1-2          | Users , Posts y una clave ajena                              | Cualquier pregunta no prevista al diseñar | Reunir una pregunta con todo lo suyo: varios JOIN |
| <b>Documentos</b> · 3-4          | La pregunta, con sus respuestas dentro                       | Mostrar una pregunta completa de una vez  | «¿Qué ha respondido Luis?», salvo duplicar        |
| <b>Familia de columnas</b> · 5-6 | Una tabla por consulta; la clave decide dónde vive cada fila | Las consultas conocidas, a gran escala    | Cualquier consulta no prevista                    |
| <b>Grafo</b> · 7                 | Nodos y aristas: Usuario , Pregunta , Respuesta              | Recorridos: quién responde a quién        | Agregados sobre todo el conjunto                  |

Ninguno es «el correcto». Guardar junto (**agregar**) o guardar aparte y enlazar (**referenciar**) es la decisión que volverá en la sesión 4

# Código verosímil sobre datos sin modelo

---

Una IA ve unos cuantos documentos de `Posts` y escribe:

```
vistas_medias = mean(p["ViewCount"] for p in posts)
sin_aceptar = [p for p in posts if not p["AcceptedAnswerId"]]
por_autor = Counter(p["OwnerId"] for p in posts)
```

Se ejecuta y devuelve números valores erróneos:

- `ViewCount` vale 0 en todas las respuestas: **la media es incorrecta**
- `AcceptedAnswerId` es nulo en las respuestas, así que se mezclan preguntas sin respuesta aceptada con respuestas (que no tienen respuesta aceptada, obviamente)
- `OwnerId` nulo reúne bajo `None` a todos los autores que borraron su cuenta

# Lo que añade el esquema

```
CREATE TABLE Posts (  
  Id INT PRIMARY KEY,  
  PostTypeId TINYINT NOT NULL, -- 1 pregunta, 2 respuesta, 4-7 wikis y otros  
  AcceptedAnswerId INT NULL, -- sólo en preguntas; NULL = sin aceptar  
  ViewCount INT NOT NULL DEFAULT 0, -- sólo en preguntas; 0 en el resto  
  OwnerUserId INT NULL, -- NULL = cuenta borrada  
  FOREIGN KEY (AcceptedAnswerId) REFERENCES Posts(Id),  
  FOREIGN KEY (OwnerUserId) REFERENCES Users(Id)  
);
```

Valores posibles, significado de `NULL` y del `0`, y qué referencia a qué: nada de eso se deduce de los datos. Lo fija el **modelo** y lo escribe el **esquema**

OJO: En este caso el modelado relacional **no es capaz de recoger todas las restricciones**

# Modelo relacional

Tablas, claves, restricciones y álgebra para consultar sin depender de cómo se almacenan físicamente los datos

# Modelo relacional

---

- Propuesto por **E. F. Codd en 1970**
- Es un modelo matemático de datos, no un producto concreto ni un lenguaje de consulta
- SQL se utiliza después para implementar ese modelo en una base de datos
- Modela los datos mediante **relaciones**, habitualmente representadas como tablas
  - Al estilo de las relaciones (funciones) que relacionan una función con su valor
- Se apoya en teoría de conjuntos, lógica de predicados y álgebra relacional
- Su fuerza es la independencia entre los datos y las aplicaciones que los usan: permite formular consultas no previstas al diseñar la base de datos

# Una relación es una tabla

---

- Una relación se ve como una **tabla de dos dimensiones**, con **columnas** y **filas**
- Todas las filas tienen **las mismas columnas**
- Cada fila se llama también **tupla**, y representa un elemento: un usuario, una pregunta
- Cada columna o **atributo** tiene un nombre y un **tipo de datos sencillo** (escalar): un número, un texto, una fecha. Nunca una lista ni otra tabla
- Los nombres de las columnas y sus tipos forman el **esquema** de la relación

# Clave primaria: identificar cada fila

---

- Cada entidad tiene identidad propia, así que cada fila debe poder **distinguirse** de las demás
- La **clave primaria** es **una o varias columnas** cuyo valor identifica de forma única cada fila
- Su valor no se repite en dos filas y no admite `NULL`
- Simple, de una columna: un usuario, por `UserId`
- Compuesta, de varias: un préstamo de la biblioteca, por ( `SocioId` , `LibroId` , `Fecha` )

# Clave ajena: relacionar entidades

---

- Las entidades se relacionan entre sí: cada *post* lo escribe un usuario
- En el modelo relacional esa conexión se expresa con una **clave ajena**: una o varias columnas de una tabla que **apuntan** a una fila de otra tabla
- Para apuntar a esa fila, la clave ajena usa **las mismas columnas** (mismo número, mismos tipos) que la **clave primaria** de la otra tabla, y guarda sus valores
- Ojo con el nombre: en el modelo relacional, **relación** es la tabla, no la conexión entre entidades

# Clave primaria y clave ajena

| Users         |       | Posts         |        |              |
|---------------|-------|---------------|--------|--------------|
| <u>UserId</u> | Name  | <u>PostId</u> | UserId | Title        |
| 1             | Diego | 1             | 1      | SQL Question |
| 38            | Pepe  | 2             | 38     | Java!        |

`Posts.UserId` apunta a `Users.UserId`: no debería existir un *post* cuyo usuario no exista

El nombre del usuario no se repite en cada *post*: cada dato está en un solo sitio y se referencia desde los demás (**normalización**), lo que evita redundancias y anomalías

# Restricciones de integridad

---

| Restricción     | Regla                                                                  |
|-----------------|------------------------------------------------------------------------|
| Clave / entidad | No hay dos filas con la misma clave primaria; su valor no es nulo      |
| Dominio         | Cada atributo solo contiene valores de su tipo y reglas asociadas      |
| Referencial     | Una clave ajena apunta a una fila existente, salvo que se permita NULL |

# Álgebra relacional

---

- El modelo relacional se definió como un **lenguaje formal** para manipular relaciones (álgebra y lógica de predicados)
- Toda búsqueda o conexión entre datos se expresa con **selección, proyección, producto cartesiano, unión y reunión** (*join*)
- Cada operación recibe relaciones y **produce otra relación**: se pueden componer y reordenar según sus propiedades algebraicas
- Nos abstrae de la representación física

# Álgebra relacional

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

select

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

project

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

join

# Selección y proyección con un ejemplo

$R$

| Id | Nombre | Reputación |
|----|--------|------------|
| 1  | Ana    | 250        |
| 2  | Luis   | 80         |
| 3  | Eva    | 1200       |

$R' = \sigma_{\text{Reputación} > 100}(R)$

| Id | Nombre | Reputación |
|----|--------|------------|
| 1  | Ana    | 250        |
| 3  | Eva    | 1200       |

$\Pi_{\text{Nombre}}(R)$

| Nombre |
|--------|
| Ana    |
| Luis   |
| Eva    |

$\sigma$  se queda con **filas**;  $\Pi$ , con **columnas**. Como el resultado es otra relación, se pueden componer y, a veces, reordenar:

$$\Pi_{\text{Nombre, Reputación}}(\sigma_{\text{Reputación} > 100}(R)) = \sigma_{\text{Reputación} > 100}(\Pi_{\text{Nombre, Reputación}}(R))$$

Sólo si la proyección conserva los atributos de la condición: tras  $\Pi_{\text{Nombre}}$  ya no hay Reputación que filtrar

# Consultas equivalentes, costes diferentes

---

El optimizador puede reordenar operaciones equivalentes para reducir el volumen de datos intermedio

$$\sigma_{p=\text{conoce}}(R) \bowtie_{o=s} \left( \sigma_{p=\text{tieneCuenta}}(R) \bowtie_{o=s} \sigma_{p=\text{web}}(R) \right)$$

$$\left( \sigma_{p=\text{conoce}}(R) \bowtie_{o=s} \sigma_{p=\text{tieneCuenta}}(R) \right) \bowtie_{o=s} \sigma_{p=\text{web}}(R)$$

$$\sigma_{p_1=\text{conoce} \wedge p_2=\text{tieneCuenta} \wedge p_3=\text{web}}(R \times R \times R)$$

No todas tienen el mismo coste, aunque describan el mismo resultado

# SQL: consultar datos

El lenguaje declarativo para expresar qué resultado queremos; el SGBD decide cómo obtenerlo

# SQL y sus dialectos

---

- SQL es un estándar ANSI/ISO; SQL:2023 incorpora, entre otras extensiones, consultas de grafos de propiedades
- Es el lenguaje que se utiliza para especificar la construcción de tablas relacionales y consultas (es una manera de escribir selección, proyección, etc.)
- Está compuesto por dos partes:
  - **DML:** (*Data Manipulation Language*) consulta y modificación de datos
  - **DDL:** (*Data Definition Language*) definición del esquema, creación de tablas, etc.
- SQL trabaja con tablas, columnas y filas; a diferencia del álgebra relacional, una consulta puede devolver duplicados (*bag semantics*)
- En esta sesión usamos el núcleo común de SQL

# Herramientas de la asignatura

---

En la asignatura utilizaremos diferentes implementaciones de SQL con pequeñas variaciones pero que tienen usos concretos

| Herramienta   | Papel                                                                                          |
|---------------|------------------------------------------------------------------------------------------------|
| <b>SQLite</b> | Primer contacto: un único fichero, consola sencilla y sin servidor                             |
| <b>MySQL</b>  | Prácticas con servidor, transacciones, índices y planes<br>EXPLAIN                             |
| <b>DuckDB</b> | Análisis de datos y lectura directa de Parquet; útil para contrastar optimizaciones analíticas |

Los ejemplos deben ejecutarse con el motor indicado: la sintaxis y, sobre todo, los planes no son idénticos

# Modo de trabajo

---

- El objetivo es manejar SQL con comodidad, no memorizar un catálogo de funciones
- Trabajaremos sobre una instantánea de datos de Stack Overflow cargada por los notebooks de la asignatura
- El mismo conjunto permite estudiar esquema, consultas, calidad de datos, relaciones, agregación y rendimiento
- El cuaderno prepara los datos desde Parquet; DuckDB puede consultarlos directamente y MySQL permite estudiarlos como base de datos de servidor

# Primer contacto con SQLite

---

SQLite almacena una base de datos relacional en un fichero y añade comandos de administración al intérprete:

```
$ sqlite3 ejemplo.db
sqlite> .tables
sqlite> .schema Posts
sqlite> .open otro-fichero.db
sqlite> .help
```

Estos comandos no son SQL: son ayudas de la consola de SQLite

# Descarga de los datos de prueba

---

Podéis descargar los datos de Stackoverflow para probar con sqlite3 ([aquí](#)):

```
$ wget https://github.com/dsevilla/bd2-data/releases/download/sqlite-backup-26-27/es.stackoverflow.db.gz
$ gunzip es.stackoverflow.db.gz
$ sqlite3 es.stackoverflow.db
sqlite>
```

# Base de datos de Stack Overflow

QuestionsTagsUsersBadgesUnansweredAsk Question

## How to remove comma at end of last line in an array

I am using `sed 's/,,$//'` to try to remove the comma at the end of the final line of an array, which has the opening and closing brackets on their lines (as shown below). This command `sed 's/,,$//'` isn't working as intended.

If I do `sed 's/,$//'` it will remove the closing array bracket, so I know I'm kind of on the right track....

```
[{"foo": "foofoo", "fuzz": ["fee, fi, fo, fum"]}, {"bar": "barbar", "fuzz": ["blah", "blah"]}]
```

this is for a bash script. please do not provide answer that uses perl.

bash sed

share edit close flag

edited 1 hour ago

asked 1 hour ago

Leahcim

8,207 27 100 196

asked today

viewed 16 times

active today

Featured on Meta

[New Navigation Release Candidate](#)

Hot Meta Posts

8 [Is it wrong to try to draw attention to a post I feel should be closed?](#)

5 [What if I think a deleted answer is worthwhile?](#)

25 [Syntax Highlighting for Neo4j / Cypher questions](#)

Looking for a job?

**Senior PHP Developer**  
**Trovit**  
Barcelona, Spain  
[php](#) [mysql](#)

**Systems Engineer**  
**Brandcrumb**  
Barcelona, Spain  
[hadoop](#) [nosql](#)

Related

[How can I remove the first](#)

Is this supposed to be valid JSON? The thing producing it should be fixed to stop adding the incorrect comma. — chepner 1 hour ago

@chepner yes, it is. not sure how to do that (subject for another question...) — Leahcim 1 hour ago

[add a comment](#)

2 Answers

active oldest votes

There is no comma in last line so `'s/,,$//'` will not work for `sed`.

1 Instead of `sed` you can use `awk`:

```
awk ' $1 == "]" {sub(/,$/, "", s)} NR>1 {print s} {s=$0} END{print s}' file
```

```
[{"foo": "foofoo", "fuzz": ["fee, fi, fo, fum"]}, {"bar": "barbar", "fuzz": ["blah", "blah"]}]
```

# Esquema de Stack Overflow

---

- **Posts:** preguntas y respuestas; `PostTypeId` distingue ambos tipos (1 = pregunta, 2 = respuesta) y `ParentId` enlaza una respuesta con su pregunta
  - Se ha tomado la decisión de albergar tanto las preguntas como las respuestas en la misma tabla `Posts`
- **Users:** usuarios y reputación
- **Comments** y **Votes:** actividad relacionada con un *post* y, cuando procede, un usuario
- **Tags** y **PostTags:** relación muchos-a-muchos entre preguntas y etiquetas

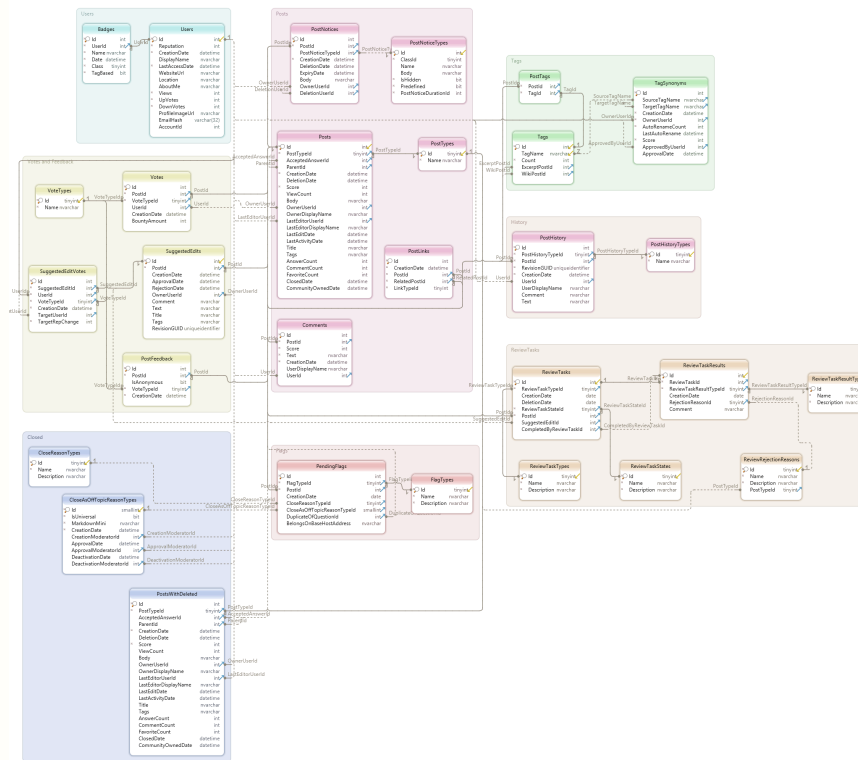
# Diagrama del esquema



# El esquema completo de Stack Exchange

SEDE incluye muchas más tablas que las cinco que usaremos en prácticas.

Fuente: «The Schema Visualized»



# El subconjunto que usaremos en prácticas

Las cinco tablas publicadas, con sus claves ajenas reales



# Lo que el diagrama no dice

---

- `Posts` se referencia a sí misma dos veces: `ParentId` enlaza una respuesta con su pregunta y `AcceptedAnswerId` la pregunta con su respuesta aceptada. Aparecen como claves ajenas, pero no se dibujan para no enredar el diagrama
- Casi todas las claves ajenas son **opcionales**: un *post* puede no tener usuario propietario porque la cuenta se borró, y un voto puede ser anónimo
- **Que una clave ajena esté declarada no significa que se cumpla**: este *dump* contiene votos y comentarios que apuntan a publicaciones ya borradas. Habrá que decidir explícitamente qué hacer con ellos antes de activar las restricciones
- El diagrama no está dibujado a mano: se genera a partir de los metadatos del propio esquema Parquet, los mismos que en la sesión 0 generaban el `CREATE TABLE`

# SELECT: forma general

---

```
SELECT lista_de_columnas  
FROM lista_de_tablas  
WHERE condicion;
```

Primero se seleccionan las filas que cumplen la condición y después se proyectan las columnas solicitadas

# Elegir columnas

---

```
SELECT * FROM Tags;
```

```
SELECT Id, Title FROM Posts;
```

- `*` es útil para explorar, pero en consultas reales conviene pedir solo las columnas necesarias
- Las comillas de identificadores no son portables: evitamos depender de ellas usando nombres sencillos y sin palabras reservadas

# DISTINCT, alias y LIMIT

---

```
SELECT DISTINCT PostTypeId FROM Posts;
```

```
SELECT DisplayName AS nombre FROM Users;
```

```
SELECT DisplayName FROM Users LIMIT 2;
```

- **DISTINCT** elimina duplicados del resultado
- **AS** nombra columnas o expresiones
- **LIMIT** facilita la exploración; no sustituye a un orden explícito

# Ordenar resultados

---

```
SELECT Id, DisplayName, Reputation  
FROM Users  
ORDER BY Reputation DESC, Id ASC;
```

Sin `ORDER BY`, SQL **no garantiza** ningún orden de salida

# Selección y proyección

| Álgebra relacional                     | SQL                     |
|----------------------------------------|-------------------------|
| Selección de filas: $\sigma_c(R)$      | SELECT * FROM R WHERE c |
| Proyección de columnas: $\Pi_{a,b}(R)$ | SELECT a, b FROM R      |

```
SELECT Id, DisplayName
FROM Users
WHERE Reputation > 100;
```

El optimizador puede adelantar proyecciones, pero debe conservar las columnas necesarias para evaluar los filtros y las uniones

# Filtrar con WHERE

```
SELECT Id, DisplayName, Reputation
FROM Users
WHERE DisplayName = 'Elenasys';
```

| Operador            | Uso                     |
|---------------------|-------------------------|
| = , <> , < , >=     | Comparaciones           |
| BETWEEN ... AND ... | Rango inclusivo         |
| IN (...)            | Pertenencia a una lista |
| LIKE                | Patrón de texto         |
| NOT , AND , OR      | Composición lógica      |

# Patrones y condiciones compuestas

---

```
SELECT DisplayName
FROM Users
WHERE Reputation > 100
      AND Location LIKE '%Spain%';
```

```
SELECT DisplayName
FROM Users
WHERE DisplayName LIKE '%sys%';
```

`AND` tiene mayor prioridad que `OR` ; usar paréntesis hace la intención inequívoca

# Expresiones y alias

---

```
SELECT DisplayName,  
       UpVotes + DownVotes AS votos_totales  
FROM Users  
WHERE UpVotes + DownVotes > 10;
```

El alias de `SELECT` no se puede usar de forma portable en `WHERE` : en esta fase de la consulta todavía no existe. Para reutilizarlo, se usa una subconsulta, una CTE o se repite la expresión

# NULL no es un valor ordinario

---

- NULL significa desconocido, ausente o no aplicable; no equivale a 0 ni a una cadena vacía
- col = NULL no encuentra filas: se usa col IS NULL o col IS NOT NULL
- Las comparaciones con NULL producen «desconocido», que WHERE no conserva

```
SELECT Id, ParentId
FROM Posts
WHERE ParentId IS NULL;
```

# Funciones de agregación

---

```
SELECT AVG(Score) AS puntuacion_media,  
       MAX(Score) AS puntuacion_maxima,  
       COUNT(*) AS numero_posts  
FROM Posts;
```

- `COUNT(*)` cuenta filas; `COUNT(columna)` solo cuenta valores no nulos
- `AVG`, `SUM`, `MIN` y `MAX` ignoran los valores nulos
- `COUNT(DISTINCT OwnerUserId)` cuenta valores distintos no nulos

# Funciones: núcleo y dialectos

| Familia          | Ejemplos portables o frecuentes         |
|------------------|-----------------------------------------|
| Numéricas        | ABS , CEIL , POWER , SQRT               |
| Cadenas          | LOWER , UPPER , TRIM , SUBSTR , REPLACE |
| Control de nulos | COALESCE , NULLIF                       |
| Condicionales    | CASE WHEN ... THEN ... ELSE ... END     |

CONCAT , funciones de desviación típica, aleatoriedad y fechas presentan diferencias entre motores; se ven en el notebook. Por ejemplo, la concatenación de cadenas es || en SQLite y DuckDB, y CONCAT( ) en MySQL

# CASE y COALESCE

---

```
SELECT DisplayName,  
       CASE  
         WHEN Reputation > 100 THEN 'Veterano'  
         ELSE 'Rookie'  
       END AS tipo,  
       COALESCE(UpVotes, 0) AS votos_positivos  
FROM Users;
```

COALESCE(a, b, ...) devuelve el primer argumento que no es nulo

# Fechas: filtrar sin perder el índice

---

Evitar, cuando se busca rendimiento:

```
WHERE YEAR(CreationDate) = 2025
```

Preferir un rango semicerrado:

```
WHERE CreationDate >= '2025-01-01'  
AND CreationDate < '2026-01-01'
```

El rango es un predicado **sargable**: no envuelve la columna, y el motor puede traducirlo a un intervalo de búsqueda en el índice. Las funciones de fecha dependen del motor, pero el principio es general; lo retomaremos con índices y `EXPLAIN` en la sesión 2

# Consultar varias tablas

Producto cartesiano, `JOIN`, agrupación y subconsultas

# Producto cartesiano

Posts

| Id | Title             | OwnerUserId |
|----|-------------------|-------------|
| 10 | Ordenar por fecha | 1           |
| 11 | Contar filas      | 1           |

Users

| Id | DisplayName |
|----|-------------|
| 1  | Ana         |
| 2  | Luis        |

Posts  $\times$  Users

| p.Id      | p.Title                  | p.OwnerUserId | u.Id     | u.DisplayName |
|-----------|--------------------------|---------------|----------|---------------|
| <b>10</b> | <b>Ordenar por fecha</b> | <b>1</b>      | <b>1</b> | <b>Ana</b>    |
| 10        | Ordenar por fecha        | 1             | 2        | Luis          |
| <b>11</b> | <b>Contar filas</b>      | <b>1</b>      | <b>1</b> | <b>Ana</b>    |
| 11        | Contar filas             | 1             | 2        | Luis          |

**Cada** *post* con **cada** usuario:  $2 \times 2 = 4$  filas. Sólo en las de negrita coincide `OwnerUserId` con `Users.Id`

# Equi-join: quedarse con las que coinciden

$$\text{Posts} \bowtie_{\text{OwnerUserId=Users.Id}} \text{Users} = \sigma_{\text{OwnerUserId=Users.Id}}(\text{Posts} \times \text{Users})$$

| p.Id | p.Title           | p.OwnerUserId | u.Id | u.DisplayName |
|------|-------------------|---------------|------|---------------|
| 10   | Ordenar por fecha | 1             | 1    | Ana           |
| 11   | Contar filas      | 1             | 1    | Ana           |

- Un **equi-join** conserva sólo las combinaciones en las que la clave ajena coincide con la clave primaria a la que apunta
- Luis no aparece: no tiene ningún *post* con el que emparejarse

# El producto cartesiano en el FROM

```
SELECT c.Id, p.Title
FROM Comments AS c, Posts AS p
WHERE p.Id = c.PostId
      AND p.Score > 10;
```

- Dos tablas en el FROM, separadas por comas, son el **producto cartesiano** Comments  $\times$  Posts: cada comentario con cada *post*
- `p.Id = c.PostId` en el WHERE es la selección  $\sigma$  que deja sólo las combinaciones que coinciden: el **equi-join**
- Sin esa condición saldrían todas las combinaciones, tengan o no que ver entre sí

# JOIN explícito

---

La misma consulta, con la condición de unión en su sitio:

```
SELECT c.Id, p.Title
FROM Comments AS c
JOIN Posts AS p ON p.Id = c.PostId
WHERE p.Score > 10;
```

- El `JOIN ... ON` hace visible la condición de unión, separada de los filtros del `WHERE`
- Evitamos la sintaxis antigua con tablas separadas por comas: es más fácil olvidar una condición y crear un producto cartesiano accidental

# USING: cuando la columna se llama igual

```
SELECT PostId, c.UserId, v.VoteTypeId  
FROM Comments AS c  
JOIN Votes AS v USING (PostId);
```

- USING (col) abrevia ON a.col = b.col, y sólo vale si la columna **se llama igual** en las dos tablas
- La columna aparece **una vez** en el resultado y se cita sin cualificar; con ON saldría dos veces y SELECT PostId sería ambiguo
- Si los nombres difieren, como Posts.Id frente a Comments.PostId, hay que seguir usando ON

Es la misma reunión, escrita sin repetir el nombre de la clave

# Preguntas y quien las hizo

---

```
SELECT u.DisplayName, p.Title
FROM Posts AS p
JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 1;
```

La misma idea se reutiliza para enlazar comentarios, votos, etiquetas y el historial con los *posts*

Recuerda que `OwnerUserId` guarda, en cada Post, el autor (clave ajena a `Users.Id` )

# LEFT JOIN: conservar filas sin coincidencia

```
SELECT u.Id, u.DisplayName, p.Id, p.Title
FROM Users AS u
LEFT JOIN Posts AS p ON p.OwnerUserId = u.Id;
```

| u.Id | u.DisplayName | p.Id | p.Title           |
|------|---------------|------|-------------------|
| 1    | Ana           | 10   | Ordenar por fecha |
| 1    | Ana           | 11   | Contar filas      |
| 2    | Luis          | NULL | NULL              |

- Conserva **todas** las filas de la tabla de la izquierda ( `Users` ), tengan o no pareja
- Luis no tiene *posts*: aparece una vez, con las columnas de `Posts` a `NULL`
- Con `JOIN` (equi-join), Luis desaparecería

# RIGHT JOIN y FULL OUTER JOIN

Añadimos el *post* 12, de una cuenta borrada ( `ownerUserId` a `NULL` ), y unimos `Users` (izquierda) con `Posts` (derecha) por `p.OwnerUserId = u.Id`:

| u.Id | u.DisplayName | p.Id | p.Title           | JOIN | LEFT | RIGHT | FULL OUTER |
|------|---------------|------|-------------------|------|------|-------|------------|
| 1    | Ana           | 10   | Ordenar por fecha | ✓    | ✓    | ✓     | ✓          |
| 1    | Ana           | 11   | Contar filas      | ✓    | ✓    | ✓     | ✓          |
| 2    | Luis          | NULL | NULL              |      | ✓    |       | ✓          |
| NULL | NULL          | 12   | Índices en MySQL  |      |      | ✓     | ✓          |

- `RIGHT JOIN` conserva todas las filas de la derecha: es un `LEFT JOIN` con las tablas intercambiadas, y se suele escribir así
- `FULL OUTER JOIN` conserva las de ambos lados. MySQL no lo tiene: se obtiene con la `UNION` de un `LEFT` y un `RIGHT JOIN`
- `OUTER` es opcional: `LEFT OUTER JOIN` es lo mismo que `LEFT JOIN`

# LEFT JOIN para buscar lo que falta

---

```
SELECT p.Id, p.Title
FROM Posts AS p
LEFT JOIN Posts AS a
  ON a.ParentId = p.Id
  AND a.PostTypeId = 2
WHERE p.PostTypeId = 1
  AND a.Id IS NULL;
```

- Preguntas **sin ninguna respuesta**: el `LEFT JOIN` las conserva con las columnas de `a` a `NULL`, y `a.Id IS NULL` se queda sólo con ellas
- `a.PostTypeId = 2` va en el `ON`, no en el `WHERE`: en el `WHERE` descartaría las filas con `a` a `NULL` y el `LEFT JOIN` se comportaría como un `JOIN`

# GROUP BY: una fila por grupo

```
SELECT OwnerUserId, COUNT(*) AS preguntas
FROM Posts
WHERE PostTypeId = 1
GROUP BY OwnerUserId;
```

Posts (preguntas)

| Id | Title             | OwnerUserId |
|----|-------------------|-------------|
| 10 | Ordenar por fecha | 1           |
| 11 | Contar filas      | 1           |
| 12 | Índices en MySQL  | NULL        |

Resultado

| OwnerUserId | preguntas |
|-------------|-----------|
| 1           | 2         |
| NULL        | 1         |

- Las filas con el mismo `OwnerUserId` forman un **grupo**, y `COUNT(*)` da un valor por grupo
- Los `NULL` forman su propio grupo. Luis no aparece: sin filas no hay grupo

# Varias funciones de agregación

```
SELECT ParentId,  
       COUNT(*) AS respuestas,  
       MAX(Score) AS mejor_puntuacion,  
       AVG(Score) AS puntuacion_media  
FROM Posts  
WHERE PostTypeId = 2  
GROUP BY ParentId;
```

- Una fila por pregunta respondida, con su número de respuestas y la puntuación de la mejor y la media
- `COUNT` , `SUM` , `AVG` , `MIN` y `MAX` resumen cada grupo en un valor
- Las preguntas sin respuestas no salen: no hay ninguna fila con su `ParentId`

## GROUP BY por dos columnas y HAVING

```
SELECT OwnerUserId, PostTypeId, COUNT(*) AS numero_posts
FROM Posts
GROUP BY OwnerUserId, PostTypeId
HAVING COUNT(*) > 10
ORDER BY OwnerUserId ASC, numero_posts DESC;
```

- WHERE filtra filas **antes** de agrupar
- HAVING filtra grupos **después** de agregar
- En SQL estricto, cada expresión seleccionada debe estar agrupada, agregada o ser funcionalmente dependiente de las columnas agrupadas

# Agrupar por la clave, mostrar el nombre

```
SELECT u.DisplayName, COUNT(*) AS respuestas
FROM Posts AS p
JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 2
GROUP BY u.Id, u.DisplayName
HAVING COUNT(*) > 100
ORDER BY respuestas DESC;
```

- Se agrupa por `u.Id` porque identifica al usuario: agrupar por `DisplayName` fundiría a dos usuarios que se llamen igual y sumaría sus respuestas
- `u.DisplayName` se añade sólo para mostrarlo: depende de `u.Id`, así que no parte ningún grupo
- Omitirlo es válido en el estándar y en MySQL 8, pero DuckDB lo rechaza y SQLite acepta incluso columnas que no dependen de la clave, con un valor cualquiera: listarlo funciona en los tres

# LEFT JOIN para contar respuestas

```
SELECT p.Id, COUNT(a.Id) AS respuestas
FROM Posts AS p
LEFT JOIN Posts AS a
  ON a.ParentId = p.Id
  AND a.PostTypeId = 2
WHERE p.PostTypeId = 1
GROUP BY p.Id;
```

- Partiendo de las preguntas, las que no tienen respuestas siguen apareciendo, con 0. Agrupando sólo las respuestas por ParentId, o con un JOIN, desaparecerían
- Se cuenta COUNT(a.Id), no COUNT(\*): la fila de una pregunta sin respuestas existe, pero su a.Id es NULL y no se cuenta

# Tablas como conjuntos

---

```
SELECT UserId FROM Comments  
UNION  
SELECT OwnerUserId FROM Posts;
```

- `UNION` elimina duplicados; `UNION ALL` los conserva
- `INTERSECT` y `EXCEPT` están disponibles en muchos motores, con diferencias de versión
- `IN` expresa pertenencia a un conjunto; con `NULL` hay que revisar cuidadosamente su semántica

# UNION ALL : apilar hechos, no deduplicarlos

```
SELECT 'pregunta' AS Tipo, Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 1
UNION ALL
SELECT 'respuesta', Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 2;
```

- UNION tiene que ordenar o construir una estructura auxiliar para descartar duplicados; UNION ALL no
- Cuando las ramas son disjuntas por construcción, UNION ALL es a la vez lo correcto y lo barato
- Todas las ramas necesitan el mismo número de columnas y tipos compatibles; los nombres los pone la primera
- Cada rama aporta su tipo como columna constante: eso es lo que distingue después a cada fila

# El riesgo: identificadores que chocan

```
SELECT 'alta_usuario' AS Tipo, Id FROM Users
UNION ALL
SELECT 'pregunta', Id FROM Posts WHERE PostTypeId = 1;
```

- `Users.Id` y `Posts.Id` son espacios de identificadores independientes, y ambos empiezan en 1
- El usuario 5 y la pregunta 5 son hechos distintos que comparten el valor de `Id`
- `PRIMARY KEY (Id)` fallaría: la clave tiene que ser `(Tipo, Id)`
- Preguntas y respuestas, en cambio, salen ambas de `Posts` y nunca chocan entre sí

# Una tabla de hechos con UNION ALL

```
INSERT INTO Hechos (Tipo, EntidadId, UserId, CreationDate)
SELECT 'alta_usuario', Id, Id, CreationDate FROM Users
UNION ALL
SELECT 'pregunta', Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 1
UNION ALL
SELECT 'respuesta', Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 2;
```

- Una tabla de hechos reúne en una sola línea de tiempo eventos de origen distinto
- PRIMARY KEY (Tipo, EntidadId) es lo que impide que el alta del usuario 5 y la pregunta 5 colisionen
- Ordenar por CreationDate responde preguntas que ninguna tabla de origen contesta por sí sola

# Subconsultas y tablas derivadas

---

```
SELECT u.DisplayName, c.numero_comentarios
FROM Users AS u
JOIN (
    SELECT UserId, COUNT(*) AS numero_comentarios
    FROM Comments
    GROUP BY UserId
) AS c ON c.UserId = u.Id;
```

Una subconsulta en `FROM` produce una tabla derivada que se puede unir y consultar como las demás

# Consultas correlacionadas

---

```
SELECT u.Id, u.DisplayName
FROM Users AS u
WHERE 100 < (
    SELECT COUNT(*)
    FROM Posts AS p
    WHERE p.OwnerUserId = u.Id
    AND p.PostTypeId = 1
);
```

La subconsulta depende de la fila externa. Un optimizador puede transformarla, pero debemos comprobar el plan y los índices antes de asumir que tendrá un buen coste

# Promedio de respuestas: una sutileza de **NULL**

```
SELECT AVG(COALESCE(a.respuestas, 0)) AS media_respuestas
FROM Posts AS p
LEFT JOIN (
    SELECT ParentId, COUNT(*) AS respuestas
    FROM Posts
    WHERE PostTypeId = 2
    GROUP BY ParentId
) AS a ON a.ParentId = p.Id
WHERE p.PostTypeId = 1;
```

Las preguntas sin respuesta salen del **LEFT JOIN** con **a.respuestas** a **NULL**, y **AVG** ignora los **NULL**: sin **COALESCE** sólo promedia las preguntas respondidas y la media queda sesgada al alza

# Consultas correctas y eficientes

El mismo resultado lógico puede requerir cantidades de trabajo muy distintas

# Leer un plan de MySQL

---

```
EXPLAIN
SELECT PostId
FROM Votes
WHERE PostId NOT IN (SELECT Id FROM Posts);
```

Empezaremos por preguntas simples:

- ¿Se recorre toda la tabla o se usa un índice?
- ¿Cuántas filas estima el optimizador?
- ¿Qué unión y qué orden de acceso ha elegido?

`EXPLAIN ANALYZE` añade medidas reales y ejecuta la consulta

# Pistas para leer un EXPLAIN

Cada fila de la salida es un acceso a una tabla. Para empezar, basta con fijarse en unas pocas columnas:

| Pregunta                    | Dónde mirar                          | Qué buscar                                                                                                                                   |
|-----------------------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| ¿Toda la tabla o un índice? | type , key                           | ALL recorre la tabla entera e index un índice entero; range , ref , eq_ref y const leen sólo las filas que casan. key a NULL : ningún índice |
| ¿Cuántas filas?             | rows                                 | Estimación, no medida. En una unión, las filas de cada tabla se examinan por cada fila de la anterior                                        |
| ¿Qué unión y en qué orden?  | Orden de las filas, id , select_type | Las filas con el mismo id forman una unión y se leen de arriba abajo. DEPENDENT SUBQUERY repite la subconsulta por cada fila externa         |
| ¿Trabajo extra?             | Extra                                | Using index : basta el índice. Using filesort o Using temporary : ordena o agrupa aparte                                                     |

# El plan del ejemplo

EXPLAIN de la consulta anterior en MySQL 8.4, con datos de ejemplo y algunas columnas omitidas:

| id | select_type  | table       | type   | key                 | rows   | Extra                   |
|----|--------------|-------------|--------|---------------------|--------|-------------------------|
| 1  | SIMPLE       | Votes       | index  | PostId              | 200408 | Using index             |
| 1  | SIMPLE       | <subquery2> | eq_ref | <auto_distinct_key> | 1      | Using where; Not exists |
| 2  | MATERIALIZED | Posts       | index  | PRIMARY             | 50635  | Using index             |

- `Votes` va primero y se recorre entera, pero sólo por su índice de `PostId`: no se lee la tabla
- La subconsulta se ejecuta una vez y su resultado se guarda ( `MATERIALIZED` ); cada voto se busca en él con `eq_ref`, una fila por clave
- `Not exists`: el `NOT IN` se ha convertido en un *anti-join*, así que basta con no encontrar el *post* de cada voto

# Para continuar

---

- Definir esquemas con DDL y restricciones
- Insertar, actualizar y borrar datos de forma segura
- Diseñar índices y contrastar planes con `EXPLAIN`
- Trabajar con JSON y decidir qué parte debe seguir siendo relacional